EchoVoice — Pseudocode

0.0 EchoVoice Main Control Module

```
BEGIN MainControl
    INITIALIZE session

    WHILE session is active DO
        exerciseSelection ← GET caregiver's exercise selection

        targetWord, selectionAccepted ← CALL
ManageExercise(exerciseSelection)

        IF selectionAccepted = TRUE THEN
            recordedSpeech, recordingCompleteFlag ← CALL
CaptureSpeech(targetWord, startRecordingFlag = TRUE)

            IF recordingCompleteFlag = TRUE THEN
                assessmentResult ← CALL
SpeechRecognitionAndAssessment(recordedSpeech, targetWord)
                // 3.4 writes the Assessment Record to D1
internally — Main does not write to D1 directly
                CALL GenerateFeedback(assessmentResult)
            ELSE
                DISPLAY "No response detected"
                // control returns to exercise selection, per
Structured English (Process 2.0)
                CONTINUE WHILE
            END IF
        ELSE
            DISPLAY "Selection rejected — please choose a valid
exercise"
            CONTINUE WHILE
        END IF

        IF SLP sends Goal Configuration / Assessment Oversight
input THEN
            slpInput ← GET input FROM SLP
            CALL ConfigureGoalsAndReview(slpInput)
        END IF
```

```
        IF learner OR caregiver ENDS session THEN
            EXIT WHILE
        END IF
    END WHILE

    sessionProgressSummary, longitudinalProgressReport ← CALL
MonitorProgress()
    DISPLAY sessionProgressSummary TO Caregiver
    MAKE longitudinalProgressReport AVAILABLE TO SLP
END MainControl
```

1.0 Manage Exercise

```
BEGIN ManageExercise(exerciseSelection)
    activeGoals ← READ D2 (Goal Config Store)

    IF exerciseSelection IS VALID AND CONSISTENT WITH
activeGoals THEN
        targetWord, targetPhonemeSequence ← DETERMINE target
word FOR exerciseSelection
        RETURN targetWord, selectionAccepted = TRUE
    ELSE
        REJECT exerciseSelection
        PROMPT Caregiver TO choose a valid exercise
        RETURN targetWord = NULL, selectionAccepted = FALSE
    END IF
END ManageExercise
```

2.0 Capture Speech

```
BEGIN CaptureSpeech(targetWord, startRecordingFlag)
    DISPLAY visualPrompt(targetWord)
    PLAY auditoryPrompt(targetWord)

    IF startRecordingFlag = TRUE THEN
        OPEN microphone stream
        WAIT FOR speech input FROM learner UNTIL timeout

        IF speech input IS RECEIVED WITHIN timeout THEN
            recordedSpeech ← RECORD speech input AS rawAudioFile
(.wav)
```

```
            recordingCompleteFlag ← TRUE
        ELSE
            FLAG no-response event
            recordedSpeech ← NULL
            recordingCompleteFlag ← FALSE
            // control returns to 1.0 Manage Exercise, per
Structured English
        END IF
        CLOSE microphone stream
    ELSE
        recordingCompleteFlag ← FALSE
    END IF

    RETURN recordedSpeech, recordingCompleteFlag
END CaptureSpeech
```

3.0 Speech Recognition & Assessment (Orchestrator)

```
BEGIN SpeechRecognitionAndAssessment(recordedSpeech, targetWord)
    rawAudioFile ← recordedSpeech

    spectrogramTensors ← CALL
ExtractAcousticFeatures(rawAudioFile)

    predictedPhonemeSequence, inferenceStatusFlag ← CALL
RunModelInference(spectrogramTensors)

    IF inferenceStatusFlag ≠ SUCCESS THEN
        RETURN assessmentResult = ERROR("Recognition failed —
please retry")
    END IF

    phonemeErrorMatrix ← CALL
AlignPhonemes(predictedPhonemeSequence, targetWord)
    // 3.3 reads D3 (Phoneme Target Dictionary) internally

    assessmentResult ← CALL
ComputePronunciationScore(phonemeErrorMatrix)
    // 3.4 writes the Assessment Record to D1 internally

    RETURN assessmentResult
```

```
END SpeechRecognitionAndAssessment
```

3.1 Extract Acoustic Features

```
BEGIN ExtractAcousticFeatures(rawAudioFile)
    audioSignal ← LOAD rawAudioFile
    audioSignal ← NORMALIZE(audioSignal)
    audioSignal ← REMOVE silence/noise (pre-emphasis, trimming)

    spectrogram ← COMPUTE mel-spectrogram(audioSignal,
frameSize, hopLength)
    spectrogramTensors ← CONVERT spectrogram TO tensor format
expected by ASR model

    RETURN spectrogramTensors
END ExtractAcousticFeatures
```

3.2 Run Model Inference

```
BEGIN RunModelInference(spectrogramTensors)
    TRY
        LOAD compressed on-device ASR model (if not already
resident in memory)
        modelOutput ← model.PREDICT(spectrogramTensors)
        predictedPhonemeSequence ← DECODE(modelOutput) INTO
phoneme sequence
        inferenceStatusFlag ← SUCCESS
    CATCH inferenceError
        predictedPhonemeSequence ← NULL
        inferenceStatusFlag ← FAILURE
    END TRY

    RETURN predictedPhonemeSequence, inferenceStatusFlag
END RunModelInference
```

3.3 Align Phonemes

```
BEGIN AlignPhonemes(predictedPhonemeSequence, targetWord)
```

```
    targetPhonemeTemplate ← READ D3 (Phoneme Target Dictionary)
USING targetWord

    alignmentMatrix ← INITIALIZE
matrix[LENGTH(targetPhonemeTemplate)+1][LENGTH(predictedPhonemeS
equence)+1]

    // Dynamic-programming alignment (edit-distance style, e.g.
Needleman-Wunsch)
    FOR i FROM 0 TO LENGTH(targetPhonemeTemplate) DO
        FOR j FROM 0 TO LENGTH(predictedPhonemeSequence) DO
            IF i = 0 THEN alignmentMatrix[i][j] ← j
            ELSE IF j = 0 THEN alignmentMatrix[i][j] ← i
            ELSE IF targetPhonemeTemplate[i] =
predictedPhonemeSequence[j] THEN
                alignmentMatrix[i][j] ← alignmentMatrix[i-1][j-
1]
            ELSE
                RECORD mismatch
                alignmentMatrix[i][j] ← 1 + MIN(
                    alignmentMatrix[i-1][j],     // deletion
                    alignmentMatrix[i][j-1],     // insertion
                    alignmentMatrix[i-1][j-1])   // substitution
            END IF
        END FOR
    END FOR

    phonemeErrorMatrix ← BACKTRACE alignmentMatrix
        TAGGING each phoneme as MATCH / SUBSTITUTION / INSERTION
/ DELETION

    RETURN phonemeErrorMatrix
END AlignPhonemes
```

3.4 Compute Pronunciation Score

```
BEGIN ComputePronunciationScore(phonemeErrorMatrix)
    totalPhonemes ← COUNT(phonemeErrorMatrix)
    errorCount ← COUNT(entries IN phonemeErrorMatrix WHERE tag ≠
MATCH)
```

```
    phonemeErrorRate ← errorCount / totalPhonemes
    accuracyScore ← (1 − phonemeErrorRate) × 100

    FOR EACH phoneme IN phonemeErrorMatrix DO
        SET phoneme.score BASED ON tag (MATCH = full credit,
SUBSTITUTION = partial, INSERTION/DELETION = none)
    END FOR

    assessmentResult ← {
        targetWord,
        accuracyScore,
        phonemeErrorRate,
        phonemeLevelBreakdown ← phonemeErrorMatrix,
        timestamp ← CURRENT_TIME()
    }

    WRITE assessmentResult TO D1 (Progress Database) AS
assessmentRecord

    RETURN assessmentResult
END ComputePronunciationScore
```

4.0 Generate Feedback

```
BEGIN GenerateFeedback(assessmentResult)
    IF assessmentResult INDICATES correct/acceptable
pronunciation THEN
        GENERATE positive visualAuditoryPrompts

    ELSE IF assessmentResult INDICATES minor errors THEN
        GENERATE corrective visualAuditoryPrompts HIGHLIGHTING
the mispronounced phoneme(s)

    ELSE
        GENERATE encouraging retry visualAuditoryPrompts
        SUGGEST simplified variant of exercise
    END IF

    SEND visualAuditoryPrompts TO Learner
    RETURN visualAuditoryPrompts
END GenerateFeedback
```

5.0 Monitor Progress

```
BEGIN MonitorProgress()
    storedProgress ← READ D1 (Progress Database)

    sessionProgressSummary ← COMPUTE FROM storedProgress FOR
current session
        (attempts this session, average accuracy, simple
takeaway)
    SEND sessionProgressSummary TO Caregiver

    longitudinalProgressReport ← COMPUTE FROM storedProgress
(trends across sessions)
        (full accuracy history, most-missed phonemes, trend
lines)
    SEND longitudinalProgressReport TO SLP

    RETURN sessionProgressSummary, longitudinalProgressReport
END MonitorProgress
```

6.0 Configure Goals & Review Assessment

```
BEGIN ConfigureGoalsAndReview(slpInput)
    IF slpInput.type = "Goal Configuration" THEN
        IF VALIDATE(slpInput.goalParameters) = TRUE THEN
            WRITE slpInput.goalParameters TO D2 (Goal Config
Store)
            RETURN CONFIRMATION "Goals updated"
        ELSE
            RETURN ERROR "Invalid goal configuration — please
review entries"
        END IF

    ELSE IF slpInput.type = "Assessment Oversight" THEN
        REVIEW assessment data
        // used for oversight / reporting only — no store update
        RETURN CONFIRMATION "Review recorded"
    END IF
END ConfigureGoalsAndReview
```